

SAP ABAP & OData Technical Specification

Bangur / Shree Cement — C&F Agent Interface

Document Version: 1.0

Date: July 2026

Prepared For: SAP Development Team

Scope: ABAP Function Modules and OData Service Exposure for C&F Agent Transactions

Table of Contents

1. Overview & Architecture
 2. Technology Stack & Prerequisites
 3. Service 1 — Check MIGO Status
 4. Service 2 — Submit MIGO (Post Goods Receipt)
 5. Service 3 — Create Delivery Instruction (DI)
 6. Service 4 — Modify Delivery Instruction (DI)
 7. Service 5 — E-Way Bill Extension
 8. Service 6 — Invoice Correction
 9. OData Service Registration & Activation
 10. Error Handling & Return Codes
 11. Security & Authorisation
 12. Testing Guidelines
 13. Appendix — Data Dictionary Objects
-

1. Overview & Architecture

1.1 Integration Pattern

The C&F Agent web interface communicates with SAP ECC/S4HANA via **SAP Gateway OData services (V2)**. Each business transaction in the UI maps to a dedicated OData **EntitySet** backed by a custom ABAP Function Module (FM) or Class Method.

C&F Web App (React)

|
| HTTP/HTTPS OData V2



SAP Gateway (ICF / /sap/opu/odata/)

|
| RFC/Function Module call



SAP ECC / S4HANA Backend

- └─ MIGO (MB01 / BAPI_GOODSMVT_CREATE)
- └─ Delivery (VL01N / BAPI_OUTB_DELIVERY_CREATE_STO)
- └─ E-Way Bill (Custom Z-Table + GSP API call)
- └─ Invoice (VF01 / BAPI BILLINGDOC_CREATEMULTIPLE)

1.2 Naming Conventions

Object Type	Prefix	Example
Function Group	ZCNF_	ZCNF_MIGO_FG
Function Module	ZCNF_	ZCNF_CHECK_MIGO
ABAP Class	ZCL_CNF_	ZCL_CNF_MIGO
OData Project	ZCNF_	ZCNF_AGENT_SRV
Data Element	ZCNF_	ZCNF_SO_NUMBER
Structure	ZCNF_S_	ZCNF_S_MIGO_REQ
Table Type	ZCNF_T_	ZCNF_T_MIGO_ITEM

1.3 Single OData Project

All six services are grouped under **one OData project** to minimise ICF node proliferation and allow shared authentication:

Service Name: ZCNF_AGENT_SRV

Namespace: ZCNF_AGENT

Base URL: /sap/opu/odata/sap/ZCNF_AGENT_SRV/

2. Technology Stack & Prerequisites

Item	Requirement
SAP Release	ECC 6.0 EhP7+ or S/4HANA 1909+
SAP Gateway	SAP Gateway 2.0 SP12+ (embedded or hub)
Transport Layer	Two-system landscape (DEV → PRD)
RFC Destination	Trusted RFC from Gateway system to Backend
ICF Service	<code>/sap/opu/odata/sap/ZCNF_AGENT_SRV</code> activated
Auth Roles	Custom role <code>ZCNF_CNF_AGENT</code>
GSP Integration	REST endpoint for E-Way Bill (see §7)

3. Service 1 — Check MIGO Status

Purpose: Retrieve the list of Sales Orders / Delivery Items that are pending MIGO posting, along with MRN numbers for already-posted items.

3.1 ABAP Function Module

Name: `ZCNF_CHECK_MIGO`

Function Group: `ZCNF_MIGO_FG`

Remote-Enabled: RFC-enabled

Import Parameters

Parameter	Type	Description
<code>IV_PLANT</code>	<code>WERKS_D</code>	Plant code (optional filter)
<code>IV_SO_NUMBER</code>	<code>VBELN</code>	Sales Order number (optional filter)
<code>IV_DATE_FROM</code>	<code>DATS</code>	Order date range from
<code>IV_DATE_TO</code>	<code>DATS</code>	Order date range to

Export Parameters

Parameter	Type	Description
<code>ET_MIGO_LIST</code>	<code>ZCNF_T_MIGO_ITEM</code>	Table of MIGO items
<code>EV_TOTAL_COUNT</code>	<code>I</code>	Total record count for pagination
<code>ES_RETURN</code>	<code>BAPIRET2</code>	Return code / message

Structure **ZCNF_S_MIGO_ITEM**

Field	Type	SAP Table/Field	Description
SO_NUMBER	VBELN	VBAK-VBELN	Sales Order number
SO_ITEM	POSNR	VBAP-POSNR	SO line item
MATERIAL	MATNR	VBAP-MATNR	Material number
MATERIAL_DESC	MAKTX	MAKT-MAKTX	Material description
BRAND_GRADE	CHAR40	Custom	Brand · Grade label
PLANT	WERKS_D	VBAP-WERKS	Plant
ORDER_QTY	MENGE_D	VBAP-KWMENG	Order quantity
UOM	VRKME	VBAP-VRKME	Unit of measure
ORDER_DATE	DATS	VBAK-AUDAT	Order creation date
DELIVERY_DATE	DATS	VBAP-EDATU	Requested delivery date
DELIVERY_NO	VBELN	LIKP-VBELN	Outbound delivery number
GR_STATUS	CHAR20	Derived	Pending / MIGO Done
MRN_NUMBER	CHAR20	MKPF-XBLNR	Material document reference
MAT_DOC	MBLNR	MKPF-MBLNR	Material document number
DO_NUMBER	CHAR20	Custom	Delivery Order number

ABAP Logic Outline

```
FUNCTION zcnf_check_migo.

" 1. Fetch Sales Orders from VBAK/VBAP within date range & plant filter
SELECT vbak~vbeln, vbap~posnr, vbap~matnr, vbap~werks,
       vbap~kwmeng, vbap~vrkme, vbap~edatu, vbak~audat
FROM vbak
INNER JOIN vbap ON vbak~vbeln = vbap~vbeln
INTO TABLE @DATA(lt_so)
WHERE vbak~audat BETWEEN @iv_date_from AND @iv_date_to
      AND ( @iv_plant = space OR vbap~werks = @iv_plant )
      AND ( @iv_so_number = space OR vbak~vbeln = @iv_so_number )
      AND vbak~auart IN ('ZOR', 'OR')    " Relevant order types
      AND vbap~abgru = space.           " Not rejected

" 2. Determine linked delivery numbers via VBFA (document flow)
SELECT vbeln_vl, posnn_vl, vbelv, posnv
FROM vbfa
INTO TABLE @DATA(lt_vbfa)
FOR ALL ENTRIES IN @lt_so
WHERE vbelv = @lt_so-vbeln
      AND posnv = @lt_so-posnr
      AND vbtyp_n = 'J'.               " Outbound delivery

" 3. Check GR posting via MKPF/MSEG for each delivery
"   Material document exists → MIGO Done; else Pending
SELECT mblnr, mjahr, xblnr, budat
FROM mkpf
INTO TABLE @DATA(lt_mkpf)
FOR ALL ENTRIES IN @lt_vbfa
WHERE xblnr = @lt_vbfa-vbeln_vl
      AND bwart = '101'.               " Goods receipt mvt type

" 4. Enrich and map into ET_MIGO_LIST
" 5. Set ES_RETURN-TYPE = 'S' on success

ENDFUNCTION.
```

4. Service 2 — Submit MIGO (Post Goods Receipt)

Purpose: Post a Goods Receipt (movement type 101) for a given delivery, creating a Material Document in SAP and returning the MRN (Material document Reference Number).

4.1 ABAP Function Module

Name: ZCNF_SUBMIT_MIGO

Function Group: ZCNF_MIGO_FG

Remote-Enabled: RFC-enabled

Import Parameters

Parameter	Type	Description
IS_HEADER	ZCNF_S_MIGO_HDR	Header data (posting date, ref doc)
IT_ITEMS	ZCNF_T_MIGO_SUBMIT	Line items to post

Structure ZCNF_S_MIGO_HDR

Field	Type	Description
DELIVERY_NO	VBELN	Outbound delivery number
POSTING_DATE	DATS	Goods receipt posting date
DOC_DATE	DATS	Document date
REFERENCE	XBLNR	External reference (MRN/Challan)

Structure ZCNF_S_MIGO_SUBMIT (line item)

Field	Type	Description
DELIVERY_ITEM	POSNR	Delivery item
MATERIAL	MATNR	Material
PLANT	WERKS_D	Plant
STORAGE_LOC	LGORT_D	Storage location
QUANTITY	MENGE_D	Actual received quantity
UOM	VRKME	Unit of measure
BATCH	CHARG_D	Batch number (if applicable)

Export Parameters

Parameter	Type	Description
EV_MAT_DOC	MBLNR	Posted material document number
EV_MAT_YEAR	GJAHR	Fiscal year
EV_MRN	CHAR20	MRN reference string
ET_RETURN	BAPIRET2T	Return messages

ABAP Logic Outline

```
FUNCTION zcnf_submit_migo.

  DATA: ls_goodsmvt_header TYPE bapi2017_gm_head_01,
        lt_goodsmvt_item   TYPE TABLE OF bapi2017_gm_item_create,
        ls_goodsmvt_code   TYPE bapi2017_gm_code,
        lv_mat_doc         TYPE mblnr,
        lv_mat_year        TYPE mjahr.

  " 1. Map header
  ls_goodsmvt_code-gm_code = '01'.           " Goods receipt for PO
  ls_goodsmvt_header-pstng_date = is_header-posting_date.
  ls_goodsmvt_header-doc_date   = is_header-doc_date.
  ls_goodsmvt_header-ref_doc_no = is_header-reference.

  " 2. Map line items (movement type 101 - GR against delivery)
  LOOP AT it_items INTO DATA(ls_item).
    APPEND VALUE #(
      mvt_ind      = 'B'           " Goods movement for delivery
      move_type    = '101'
      plant        = ls_item-plant
      stge_loc     = ls_item-storage_loc
      material      = ls_item-material
      entry_qnt    = ls_item-quantity
      entry_uom    = ls_item-uom
      batch        = ls_item-batch
      deliv_numb   = is_header-delivery_no
      deliv_item   = ls_item-delivery_item
    ) TO lt_goodsmvt_item.
  ENDLOOP.

  " 3. Post via standard BAPI
  CALL FUNCTION 'BAPI_GOODSMVT_CREATE'
    EXPORTING
      goodsmvt_header = ls_goodsmvt_header
      goodsmvt_code   = ls_goodsmvt_code
    IMPORTING
      materialdocument = lv_mat_doc
      matdocumentyear  = lv_mat_year
    TABLES
      goodsmvt_item    = lt_goodsmvt_item
      return           = et_return.
```



```

" 4. Check return messages for errors

READ TABLE et_return WITH KEY type = 'E' TRANSPORTING NO FIELDS.

IF sy-subrc = 0.
    CALL FUNCTION 'BAPI_TRANSACTION_ROLLBACK'.
    RETURN.
ENDIF.

CALL FUNCTION 'BAPI_TRANSACTION_COMMIT'
    EXPORTING wait = 'X'.

ev_mat_doc  = lv_mat_doc.
ev_mat_year = lv_mat_year.
ev_mrn      = |MRN-{ lv_mat_doc }|.

ENDFUNCTION.

```

5. Service 3 — Create Delivery Instruction (DI)

Purpose: Create an Outbound Delivery for a Sales Order. In the cement C&F context, a Delivery Instruction (DI) triggers the picking/packing and dispatch process.

5.1 ABAP Function Module

Name: ZCNF_CREATE_DI

Function Group: ZCNF_DELIVERY_FG

Remote-Enabled: RFC-enabled

Import Parameters

Parameter	Type	Description
IS_DI_HEADER	ZCNF_S_DI_HDR	DI header data
IT_DI_ITEMS	ZCNF_T_DI_ITEM	DI line items

Structure **ZCNF_S_DI_HDR**

Field	Type	Description
SO_NUMBER	VBELN	Reference Sales Order
SHIP_POINT	VSTEL	Shipping point
PLANNED_GI_DATE	DATS	Planned goods issue date
DELIVERY_DATE	DATS	Requested delivery date
VEHICLE_NO	CHAR20	Truck / vehicle number
DRIVER_NAME	CHAR40	Driver name
TRANSPORTER	CHAR40	Transporter name

Structure **ZCNF_S_DI_ITEM**

Field	Type	Description
SO_ITEM	POSNR	SO line item reference
MATERIAL	MATNR	Material
DELIVERY_QTY	MENGE_D	Quantity to dispatch
UOM	VRKME	Unit of measure
PLANT	WERKS_D	Plant
STORAGE_LOC	LGORT_D	Storage location
BATCH	CHARG_D	Batch number

Export Parameters

Parameter	Type	Description
EV_DELIVERY_NO	VBELN	Created delivery document number
ET_RETURN	BAPIRET2T	Return messages

ABAP Logic Outline

```
FUNCTION zcnf_create_di.

    DATA: ls_header_data      TYPE bapiibdlvhdrchg,
           ls_header_control   TYPE bapiibdlvhdrctrlchg,
           lt_item_data        TYPE TABLE OF bapiibdlvitemchg,
           lt_item_control     TYPE TABLE OF bapiibdlvitemctrlchg,
           lv_delivery         TYPE vbeln_vl.

    " 1. Populate header
    ls_header_data-ship_point   = is_di_header-ship_point.
    ls_header_data-req_dlv_date = is_di_header-delivery_date.
    ls_header_data-gds_issue_date = is_di_header-planned_gi_date.
    " Custom fields for vehicle/transporter stored in Z-Table
    " ZCNF_DI_ADDL via a separate update after delivery creation

    " 2. Map line items
    LOOP AT it_di_items INTO DATA(ls_item).
        APPEND VALUE #(
            ref_doc      = is_di_header-so_number
            ref_item      = ls_item-so_item
            deliv_qty     = ls_item-delivery_qty
            dlv_qty_imunit = ls_item-delivery_qty
            material      = ls_item-material
            plant         = ls_item-plant
            stge_loc      = ls_item-storage_loc
            batch         = ls_item-batch
        ) TO lt_item_data.
    ENDLOOP.

    " 3. Create delivery via BAPI
    CALL FUNCTION 'BAPI_OUTB_DELIVERY_CREATE_STO'
        EXPORTING
            ship_point   = is_di_header-ship_point
            due_date     = is_di_header-delivery_date
            sales_order  = is_di_header-so_number
        IMPORTING
            delivery     = lv_delivery
        TABLES
            return       = et_return.

    READ TABLE et_return WITH KEY type = 'E' TRANSPORTING NO FIELDS.
```

```

IF sy-subrc = 0.
    CALL FUNCTION 'BAPI_TRANSACTION_ROLLBACK'.
    RETURN.
ENDIF.

CALL FUNCTION 'BAPI_TRANSACTION_COMMIT'
    EXPORTING wait = 'X'.

" 4. Store vehicle/driver/transporter in Z-Table ZCNF_DI_ADDL
INSERT zcnf_di_addl FROM VALUE #(
    delivery_no   = lv_delivery
    vehicle_no    = is_di_header-vehicle_no
    driver_name   = is_di_header-driver_name
    transporter   = is_di_header-transporter
    created_by    = sy-uname
    created_at    = sy-datum
).
COMMIT WORK.

ev_delivery_no = lv_delivery.

ENDFUNCTION.

```

6. Service 4 — Modify Delivery Instruction (DI)

Purpose: Update an existing Outbound Delivery — change quantities, vehicle number, or other logistical details before goods issue.

6.1 ABAP Function Module

Name: ZCNF_MODIFY_DI

Function Group: ZCNF_DELIVERY_FG

Remote-Enabled: RFC-enabled

Import Parameters

Parameter	Type	Description
IV_DELIVERY_NO	VBELN	Delivery to modify
IS_HEADER_CHANGE	ZCNF_S_DI_HDR_CHG	Changed header fields
IT_ITEM_CHANGE	ZCNF_T_DI_ITEM_CHG	Changed line items

Structure ZCNF_S_DI_HDR_CHG

Field	Type	Description
VEHICLE_NO	CHAR20	Updated vehicle number
DRIVER_NAME	CHAR40	Updated driver name
TRANSPORTER	CHAR40	Updated transporter
PLANNED_GI_DATE	DATS	Updated planned GI date

Structure ZCNF_S_DI_ITEM_CHG

Field	Type	Description
DELIVERY_ITEM	POSNR	Delivery item to change
DELIVERY_QTY	MENGE_D	New delivery quantity
BATCH	CHARG_D	Updated batch

Export Parameters

Parameter	Type	Description
ET_RETURN	BAPIRET2T	Return messages

ABAP Logic Outline

```
FUNCTION zcnf_modify_di.

    DATA: ls_header_data      TYPE bapiibdlvhdrchg,
           ls_header_control  TYPE bapiibdlvhdrctrlchg,
           lt_item_data       TYPE TABLE OF bapiibdlvitemchg,
           lt_item_control    TYPE TABLE OF bapiibdlvitemctrlchg.

    " 1. Pre-check: delivery must not be goods-issued already
    SELECT SINGLE wbstk INTO @DATA(lv_gi_status)
      FROM likp WHERE vbeln = @iv_delivery_no.
    IF lv_gi_status = 'C'.
        APPEND VALUE #(
            type      = 'E'
            message = 'Delivery already goods-issued; modification not allowed.'
        ) TO et_return.
        RETURN.
    ENDIF.

    " 2. Header changes (GI date)
    ls_header_data-gds_issue_date      = is_header_change-planned_gi_date.
    ls_header_control-gds_issue_date_flg = 'X'.

    " 3. Item changes (quantity, batch)
    LOOP AT it_item_change INTO DATA(ls_chg).
        APPEND VALUE #(
            deliv_numb   = iv_delivery_no
            deliv_item   = ls_chg-delivery_item
            deliv_qty    = ls_chg-delivery_qty
            batch        = ls_chg-batch
        ) TO lt_item_data.
        APPEND VALUE #(
            deliv_numb   = iv_delivery_no
            deliv_item   = ls_chg-delivery_item
            deliv_qty_flg = 'X'
            batch_flg    = 'X'
        ) TO lt_item_control.
    ENDLOOP.

    " 4. Call change BAPI
    CALL FUNCTION 'BAPI_OUTB_DELIVERY_CHANGE'
      EXPORTING
```

```

        delivery      = iv_delivery_no
        header_data    = ls_header_data
        header_control = ls_header_control
TABLES
        item_data      = lt_item_data
        item_control    = lt_item_control
        return          = et_return.

READ TABLE et_return WITH KEY type = 'E' TRANSPORTING NO FIELDS.
IF sy-subrc = 0.
    CALL FUNCTION 'BAPI_TRANSACTION_ROLLBACK'.
    RETURN.
ENDIF.

CALL FUNCTION 'BAPI_TRANSACTION_COMMIT'
    EXPORTING wait = 'X'.

" 5. Update Z-Table for vehicle/transporter changes
UPDATE zcnf_di_addl
    SET vehicle_no = is_header_change-vehicle_no
        driver_name = is_header_change-driver_name
        transporter = is_header_change-transporter
        changed_by  = sy-uname
        changed_at  = sy-datum
    WHERE delivery_no = iv_delivery_no.
COMMIT WORK.

ENDFUNCTION.

```

7. Service 5 — E-Way Bill Extension

Purpose: Extend the validity of an active E-Way Bill via integration with the GSP (GST Suvidha Provider) REST API. The ABAP program calls the GSP endpoint, updates the new validity date, and records the extension in a Z-Table.

7.1 ABAP Function Module

Name: ZCNF_EXTEND_EWAYBILL

Function Group: ZCNF_EWAYBILL_FG

Remote-Enabled: RFC-enabled

Import Parameters

Parameter	Type	Description
IV_EWB_NUMBER	CHAR20	E-Way Bill number
IV_VEHICLE_NO	CHAR20	Updated vehicle number
IV_REASON	CHAR1	Extension reason code (1–6)
IV_REASON_REMARKS	CHAR100	Free-text reason
IV_FROM_PLACE	CHAR50	Current location (place)
IV_FROM_STATE	NUMC2	State code of current location
IV_TRANSPORT_MODE	CHAR1	Transport mode (1=Road, 2=Rail)

Extension Reason Codes

Code	Description
1	Natural Calamity
2	Law and Order situation
3	Transshipment
4	Accident
5	Others
6	First Extension

Export Parameters

Parameter	Type	Description
EV_NEW_VALIDITY	DATS	New validity date returned by GSP
EV_EWB_RESPONSE	STRING	Raw JSON response from GSP
ES_RETURN	BAPIRET2	Return code / message

Z-Table: **ZCNF_EWB_LOG**

Field	Type	Key	Description
EWB_NUMBER	CHAR20	✓	E-Way Bill number
EXTENSION_SEQ	NUMC2	✓	Extension sequence (01, 02...)
REQUEST_DATE	DATS		Date of extension request
OLD_VALIDITY	DATS		Previous validity date
NEW_VALIDITY	DATS		New validity date
VEHICLE_NO	CHAR20		Vehicle number at extension
REASON_CODE	CHAR1		Reason code
REASON_TEXT	CHAR100		Reason text
GSP_RESPONSE	STRING		Raw JSON from GSP
REQUESTED_BY	UNAME		SAP user

ABAP Logic Outline

```
FUNCTION zcnf_extend_ewaybill.
```

```
DATA: lv_url          TYPE string,  
      lv_json_body    TYPE string,  
      lv_response     TYPE string,  
      lv_http_code    TYPE i,  
      lo_http_client  TYPE REF TO if_http_client.
```

```
" 1. Build JSON request body for GSP API
```

```
lv_json_body = |{ "ewbNo": "{ iv_ewb_number }",| &  
               |"vehicleNo": "{ iv_vehicle_no }",| &  
               |"fromPlace": "{ iv_from_place }",| &  
               |"fromState": { iv_from_state },| &  
               |"remainAddr": "",| &  
               |"transMode": "{ iv_transport_mode }",| &  
               |"extnRsnCode": { iv_reason },| &  
               |"extnRemarks": "{ iv_reason_remarks }"}|.
```

```
" 2. Retrieve GSP endpoint and auth token from secure Z-Table
```

```
SELECT SINGLE endpoint, auth_token  
FROM zcnf_gsp_config  
INTO (@DATA(lv_endpoint), @DATA(lv_token))  
WHERE config_key = 'EWB_EXTEND'.
```

```
" 3. Create HTTP client and POST
```

```
cl_http_client=>create_by_url(  
    EXPORTING url          = lv_endpoint  
    IMPORTING client       = lo_http_client  
    EXCEPTIONS argument_not_found = 1 OTHERS = 2 ).
```

```
lo_http_client->request->set_method( 'POST' ).
```

```
lo_http_client->request->set_header_field(  
    name  = 'Authorization'  
    value = |Bearer { lv_token }| ).
```

```
lo_http_client->request->set_header_field(  
    name  = 'Content-Type'  
    value = 'application/json' ).
```

```
lo_http_client->request->set_cdata( lv_json_body ).
```

```
lo_http_client->send( ).
```

```
lo_http_client->receive( ).
```

```

lv_http_code = lo_http_client->response->get_status( )-code.
lv_response  = lo_http_client->response->get_cdata( ).
lo_http_client->close( ).

" 4. Parse JSON response - extract new validity date
" Use /ui2/cl_json or custom parser
DATA(lo_json) = NEW /ui2/cl_json( ).
" ... extract validUpto field into ev_new_validity ...

IF lv_http_code = 200.
    " 5. Log to ZCNF_EWB_LOG
    INSERT zcnf_ewb_log FROM VALUE #(
        ewb_number      = iv_ewb_number
        request_date    = sy-datum
        new_validity     = ev_new_validity
        vehicle_no      = iv_vehicle_no
        reason_code      = iv_reason
        reason_text      = iv_reason_remarks
        gsp_response     = lv_response
        requested_by     = sy-uname
    ).
    COMMIT WORK.
    es_return-type = 'S'.
    es_return-message = |EWB extended. New validity: { ev_new_validity }|.
ELSE.
    es_return-type = 'E'.
    es_return-message = |GSP API error { lv_http_code }: { lv_response }|.
ENDIF.

ev_ewb_response = lv_response.

ENDFUNCTION.

```

Note: GSP credentials (`endpoint` , `auth_token`) must be stored in a secure configuration Z-Table (`ZCNF_GSP_CONFIG`) and **never** hardcoded. Token refresh logic should be handled by a separate scheduled job using `ZCNF_GSP_TOKEN_REFRESH` .

8. Service 6 — Invoice Correction

Purpose: Create a Credit/Debit Memo request or reverse a billing document to correct invoice values. Used when there is a pricing discrepancy or quantity short-shipment.

8.1 ABAP Function Module

Name: ZCNF_INVOICE_CORRECTION
Function Group: ZCNF_INVOICE_FG
Remote-Enabled: RFC-enabled

Import Parameters

Parameter	Type	Description
IV_BILLING_DOC	VBELN	Original billing document
IV_CORRECTION_TYPE	CHAR1	C = Credit memo, D = Debit memo, R = Reversal
IV_REASON_CODE	CHAR3	Reason code (maps to ABGRU / order reason)
IV_REMARKS	CHAR200	Free-text correction remarks
IT_ITEMS	ZCNF_T_INV_CORR_ITEM	Line-item corrections (optional)

Structure ZCNF_S_INV_CORR_ITEM

Field	Type	Description
BILLING_ITEM	POSNR	Billing document item
CORRECT_QTY	MENGE_D	Corrected quantity (if qty dispute)
CORRECT_PRICE	KBETR	Corrected unit price (if price dispute)
REMARKS	CHAR100	Item-level remark

Export Parameters

Parameter	Type	Description
EV_CREDIT_MEMO_NO	VBELN	Created credit/debit memo request number
EV_CANCEL_DOC	VBELN	Cancellation billing document (for reversal)
ET_RETURN	BAPIRET2T	Return messages

ABAP Logic Outline

```
FUNCTION zcnf_invoice_correction.

CASE iv_correction_type.

    WHEN 'R'. " Full reversal via cancellation
        CALL FUNCTION 'BAPI_BILLINGDOC_CANCEL1'
            EXPORTING
                billingdocument = iv_billing_doc
            IMPORTING
                canceledocument = ev_cancel_doc
            TABLES
                return = et_return.

        READ TABLE et_return WITH KEY type = 'E' TRANSPORTING NO FIELDS.
        IF sy-subrc <> 0.
            CALL FUNCTION 'BAPI_TRANSACTION_COMMIT'
                EXPORTING wait = 'X'.
        ELSE.
            CALL FUNCTION 'BAPI_TRANSACTION_ROLLBACK'.
        ENDIF.

    WHEN 'C' OR 'D'. " Credit or Debit memo request
        DATA: lt_create_input  TYPE TABLE OF bapivbrkrequest,
              lt_create_output TYPE TABLE OF bapivbrkout.

        " Determine order type: CR for credit, DR for debit
        DATA(lv_order_type) = COND #(
            WHEN iv_correction_type = 'C' THEN 'CR'
            ELSE 'DR' ).

        APPEND VALUE #(
            ref_doc      = iv_billing_doc
            req_type      = lv_order_type
            reason        = iv_reason_code
        ) TO lt_create_input.

        CALL FUNCTION 'BAPI_BILLINGDOC_CREATEMULTIPLE'
            TABLES
                billingdatain = lt_create_input
                billingdataout = lt_create_output
                return = et_return.
```

```

    READ TABLE et_return WITH KEY type = 'E' TRANSPORTING NO FIELDS.

    IF sy-subrc <> 0.
        CALL FUNCTION 'BAPI_TRANSACTION_COMMIT'
            EXPORTING wait = 'X'.

        READ TABLE lt_create_output INDEX 1 INTO DATA(ls_out).
        ev_credit_memo_no = ls_out-billingdoc.
    ELSE.
        CALL FUNCTION 'BAPI_TRANSACTION_ROLLBACK'.
    ENDIF.

ENDCASE.

" Log correction in Z-Table ZCNF_INV_CORR_LOG
INSERT zcnf_inv_corr_log FROM VALUE #(
    billing_doc      = iv_billing_doc
    correction_type   = iv_correction_type
    credit_memo_no    = ev_credit_memo_no
    cancel_doc        = ev_cancel_doc
    reason_code       = iv_reason_code
    remarks           = iv_remarks
    requested_by      = sy-uname
    request_date      = sy-datum
).

COMMIT WORK.

ENDFUNCTION.

```

9. OData Service Registration & Activation

All six FMs are exposed through a **single SAP Gateway OData project** named `ZCNF_AGENT_SRV`.

9.1 Step-by-Step: Create the OData Project

Step 1 — Create the Service in SEGW

1. Navigate to **Transaction SEGW** (Service Builder).
2. Click **Create Project**.

- **Project:** `ZCNF_AGENT_SRV`
- **Description:** `Bangur C&F Agent OData Services`
- **Project Type:** `Service with SAP Annotations`

3. Click **Save**.

Step 2 — Define Data Model (Entity Types)

For each service, create an **Entity Type** in the Data Model node.

Entity Type	Entity Set	HTTP Methods
MigoItem	MigoSet	GET (list), GET (single)
MigoPost	MigoPostSet	POST
DeliveryInstruction	DeliverySet	POST
DeliveryChange	DeliveryChangeSet	PATCH / PUT
EwayBillExtension	EwayBillSet	POST
InvoiceCorrection	InvoiceCorrectionSet	POST

Example — Entity Type **MigoItem** :

Property	EDM Type	Max Length	Key
SoNumber	Edm.String	10	✓
SoItem	Edm.String	6	✓
Material	Edm.String	18	
MaterialDesc	Edm.String	40	
BrandGrade	Edm.String	40	
Plant	Edm.String	4	
OrderQty	Edm.Decimal	Precision 13, Scale 3	
Uom	Edm.String	3	
OrderDate	Edm.DateTime		
DeliveryDate	Edm.DateTime		
DeliveryNo	Edm.String	10	
GrStatus	Edm.String	20	
MrnNumber	Edm.String	20	
MatDoc	Edm.String	10	
DoNumber	Edm.String	20	

Step 3 — Generate Runtime Objects

1. Right-click the project → **Generate Runtime Objects**.
2. Specify the **Package** (e.g., **ZCNF_AGENT**) and **Transport Request**.
3. This generates:

- MPC class: `ZCL_ZCNF_AGENT_SRV_MPC`
- MPC Extension: `ZCL_ZCNF_AGENT_SRV_MPC_EXT`
- DPC class: `ZCL_ZCNF_AGENT_SRV_DPC`
- DPC Extension: `ZCL_ZCNF_AGENT_SRV_DPC_EXT`

Step 4 — Implement DPC Extension Methods

All business logic is wired in the **DPC Extension class** (`ZCL_ZCNF_AGENT_SRV_DPC_EXT`).

Example — `MIGOSET_GET_ENTITYSET` (GET list):


```

METHOD migoset_get_entityset.

    DATA: lt_migo_items TYPE TABLE OF zcnf_s_migo_item.

    " Extract filter values from IT_FILTER_SELECT_OPTIONS
    DATA(lv_plant)  = VALUE #( it_filter_select_options[
        name = 'PLANT' ]-select_options[ 1 ]-low DEFAULT space ).
    DATA(lv_so)     = VALUE #( it_filter_select_options[
        name = 'SONUMBER' ]-select_options[ 1 ]-low DEFAULT space ).

    " Call ABAP Function Module
    CALL FUNCTION 'ZCNF_CHECK_MIGO'
        EXPORTING
            iv_plant      = lv_plant
            iv_so_number  = lv_so
        IMPORTING
            et_migo_list = lt_migo_items
            ev_total_count = es_response_context-count.

    " Map ABAP structure to OData entity structure
    LOOP AT lt_migo_items INTO DATA(ls_item).
        APPEND VALUE #(
            so_number      = ls_item-so_number
            so_item        = ls_item-so_item
            material       = ls_item-material
            material_desc  = ls_item-material_desc
            brand_grade    = ls_item-brand_grade
            plant          = ls_item-plant
            order_qty      = ls_item-order_qty
            uom            = ls_item-uom
            order_date     = ls_item-order_date
            delivery_date  = ls_item-delivery_date
            delivery_no    = ls_item-delivery_no
            gr_status      = ls_item-gr_status
            mrn_number     = ls_item-mrn_number
            mat_doc        = ls_item-mat_doc
            do_number      = ls_item-do_number
        ) TO et_entityset.
    ENDLOOP.
ENDMETHOD.

```

Example — MIGOPOSTSET_CREATE_ENTITY (POST — Submit MIGO):

```

METHOD migopostset_create_entity.

  DATA: ls_req TYPE zcnf_s_migo_submit_req,
         lt_items TYPE zcnf_t_migo_submit,
         lt_return TYPE bapiret2t.

  " Deserialise incoming OData request body
  io_data_provider->read_entry_data( IMPORTING es_data = ls_req ).

  " Call ABAP FM
  CALL FUNCTION 'ZCNF_SUBMIT_MIGO'
    EXPORTING
      is_header = VALUE #(
        delivery_no  = ls_req-delivery_no
        posting_date = ls_req-posting_date
        doc_date     = ls_req-doc_date
        reference    = ls_req-reference )
      it_items = ls_req-items
    IMPORTING
      ev_mat_doc = er_entity-mat_doc
      ev_mrn     = er_entity-mrn_number
    TABLES
      et_return = lt_return.

  " Raise OData error if SAP returned errors
  READ TABLE lt_return WITH KEY type = 'E' INTO DATA(ls_error).
  IF sy-subrc = 0.
    RAISE EXCEPTION TYPE /iwbsp/cx_mgw_busi_exception
      EXPORTING
        message = ls_error-message.
  ENDIF.
ENDMETHOD.

```

Step 5 — Register and Activate in SAP Gateway

1. Navigate to **Transaction** `/n/IWFND/MAINT_SERVICE` (on Gateway system).
2. Click **Add Service**.
3. Search for `ZCNF_AGENT_SRV`.
4. Select the service, assign to the **System Alias** (backend RFC).
5. Click **Add Selected Services** → assign to package and transport.
6. Tick **Enable** checkbox → **Save**.

Step 6 — Activate ICF Node

1. Navigate to **Transaction SICF**.
2. Path: `/sap/opu/odata/sap/ZCNF_AGENT_SRV`
3. Right-click → **Activate Service**.

Step 7 — Test via Gateway Client

- Navigate to **Transaction** `/n/IWFND/GW_CLIENT`
- Test URL: `GET /sap/opu/odata/sap/ZCNF_AGENT_SRV/MigoSet?$top=10`

10. Error Handling & Return Codes

All Function Modules follow a uniform error return pattern using `BAPIRET2` / `BAPIRET2T`.

10.1 Standard Return Structure

```
" Always check for errors before COMMIT
READ TABLE et_return WITH KEY type = 'E' TRANSPORTING NO FIELDS.
IF sy-subrc = 0.
    CALL FUNCTION 'BAPI_TRANSACTION_ROLLBACK'.
    " Surface error to OData caller
ELSE.
    CALL FUNCTION 'BAPI_TRANSACTION_COMMIT' EXPORTING wait = 'X'.
ENDIF.
```

10.2 HTTP Status Mapping

BAPIRET2 Type	HTTP Status	OData Error Code
<code>S</code> (Success)	200 / 201	—
<code>W</code> (Warning)	200 + warning message	<code>CNF_WARN</code>
<code>E</code> (Error)	400	<code>CNF_BUSI_ERR</code>
Exception (auth)	403	<code>CNF_AUTH_ERR</code>
Exception (not found)	404	<code>CNF_NOT_FOUND</code>

10.3 Custom Exception Class

Create `ZCX_CNF_AGENT` inheriting from `/IWBEP/CX_MGW_BUSI_EXCEPTION` for business error propagation.

11. Security & Authorisation

11.1 Custom Authority Objects

Auth Object	Description	Fields
ZCNF_MIGO	MIGO operations	ACTVT (16=execute, 01=submit)
ZCNF_DI	Delivery instruction operations	ACTVT, WERKS
ZCNF_EWB	E-Way Bill operations	ACTVT
ZCNF_INV	Invoice correction	ACTVT

11.2 Authority Checks in FMs

```
AUTHORITY-CHECK OBJECT 'ZCNF_MIGO'
  ID 'ACTVT' FIELD '01'.    " Submit
IF sy-subrc <> 0.
  MESSAGE e001(zcnf) RAISING not_authorized.
ENDIF.
```

11.3 OData Authentication

- Use **OAuth 2.0** (preferred for external web apps) via SAP Cloud Platform or on-premise OAuth server.
- Alternatively use **Basic Auth over HTTPS** for internal network calls.
- Configure in SICF → **Logon Data** tab of the ICF node.
- Assign role ZCNF_CNF_AGENT to all C&F agent users.

12. Testing Guidelines

12.1 Unit Testing (SE37)

Test each FM individually in SE37 with representative input sets:

Test Case	FM	Expected Result
Valid SO range, plant filter	ZCNF_CHECK_MIGO	Returns matching items
Invalid delivery (GI done)	ZCNF_SUBMIT_MIGO	Error in ET_RETURN
Valid delivery creation	ZCNF_CREATE_DI	Returns EV_DELIVERY_NO
Modify after GI	ZCNF_MODIFY_DI	Error: already goods-issued
EWB extension — GSP success	ZCNF_EXTEND_EWAYBILL	New validity date returned
EWB extension — GSP timeout	ZCNF_EXTEND_EWAYBILL	Error with HTTP code
Invoice credit memo	ZCNF_INVOICE_CORRECTION	Credit memo number returned
Invoice reversal	ZCNF_INVOICE_CORRECTION	Cancel doc number returned

12.2 OData Endpoint Testing

Use **SAP Gateway Client** (`/n/IWFND/GW_CLIENT`) or **Postman**:

```
# Check MIGO list
GET /sap/opu/odata/sap/ZCNF_AGENT_SRV/MigoSet?$filter=Plant eq '1001'&$format=json

# Submit MIGO
POST /sap/opu/odata/sap/ZCNF_AGENT_SRV/MigoPostSet
Content-Type: application/json
{
  "DeliveryNo": "0080012345",
  "PostingDate": "/Date(1751414400000)/",
  "DocDate": "/Date(1751414400000)/",
  "Reference": "MRN-CHALLAN-001"
}

# Create DI
POST /sap/opu/odata/sap/ZCNF_AGENT_SRV/DeliverySet
Content-Type: application/json
{
  "SoNumber": "0000100021",
  "ShipPoint": "D001",
  "VehicleNo": "UP32AB1234",
  "DriverName": "Ramesh Kumar"
}

# Extend E-Way Bill
POST /sap/opu/odata/sap/ZCNF_AGENT_SRV/EwayBillSet
Content-Type: application/json
{
  "EwbNumber": "431000000001",
  "VehicleNo": "UP32AB1234",
  "ReasonCode": "3",
  "FromPlace": "Deoria",
  "FromState": "09"
}
```

13. Appendix — Data Dictionary Objects

13.1 Custom Z-Tables Summary

Table	Purpose
ZCNF_DI_ADDL	Additional DI fields (vehicle, driver, transporter)
ZCNF_EWB_LOG	E-Way Bill extension audit log
ZCNF_INV_CORR_LOG	Invoice correction audit log
ZCNF_GSP_CONFIG	GSP API endpoint & token config (secure)

13.2 Transport Strategy

Transport Type	Objects
Workbench	ABAP programs, classes, FMs, structures, data elements
Customising	SAP Gateway service registration, ICF activation

Transport order sequence:

- 1. Data Dictionary objects (structures, table types, Z-tables)
- 2. Function Groups and Function Modules
- 3. ABAP Classes (MPC, DPC, Exception)
- 4. OData project (SEGW artefacts)
- 5. Gateway registration + ICF activation

13.3 ABAP Package Structure

```
ZCNF_AGENT                                ← Main package
├── ZCNF_AGENT_DDIC                        ← Data dictionary objects
│   ├── Structures: ZCNF_S_*
│   ├── Table Types: ZCNF_T_*
│   └── Z-Tables: ZCNF_DI_ADDL, ZCNF_EWB_LOG, ...
├── ZCNF_AGENT_FM                          ← Function Groups & Modules
│   ├── ZCNF_MIGO_FG
│   ├── ZCNF_DELIVERY_FG
│   ├── ZCNF_EWAYBILL_FG
│   └── ZCNF_INVOICE_FG
├── ZCNF_AGENT_GATEWAY                    ← OData project artefacts
│   ├── ZCL_ZCNF_AGENT_SRV_MPC_EXT
│   └── ZCL_ZCNF_AGENT_SRV_DPC_EXT
└── ZCNF_AGENT_AUTH                      ← Auth objects & roles
    ├── Auth Objects: ZCNF_MIGO, ZCNF_DI, ...
    └── Role: ZCNF_CNF_AGENT
```

End of Document